

APIVerify

Integration Handbook

Meridian Quality Systems
Product Documentation · Version 2.3 · Intermediate Guide

About This Document

Difficulty	★★ Intermediate
Best for	Chunking, Chunk Overlap, Semantic Retrieval, Multi-Chunk Synthesis
Approximate Chunks	45
Estimated Embeddings	45
Recommended RAG Stages	Chunking → Chunk Overlap → Semantic Retrieval

This handbook uses longer paragraphs and cross-page explanations, so some questions require retrieving and combining more than one chunk to answer correctly.

Table of Contents

1. Introduction to APIVerify
2. Core Concepts
3. Installation and Setup
4. Verification Modes
5. Baseline Sync
6. Drift Detection
7. Configuration Reference
8. Integration with MockBridge and FalconTest
9. Reporting and Dashboards
10. Best Practices
11. Glossary
12. Questions to Try

1. Introduction to APIVerify

APIVerify is a contract-testing framework built by Meridian Quality Systems to validate that a REST API's real behavior continues to match the contract its consumers were promised. Where FalconTest is concerned with driving end-to-end test execution across browsers and services, APIVerify focuses narrowly on one question: does this endpoint, right now, return what the contract says it should return? That narrower scope is what lets APIVerify run quickly and frequently — often on every commit — rather than only in scheduled regression windows.

A contract, in APIVerify terminology, is a structured description of an endpoint's expected request shape, response shape, status codes, and headers. Contracts are written once by the team that owns an API and are then continuously checked against the live service. When a live response no longer matches its contract, APIVerify calls this **drift**, and the process of catching it is called **Drift Detection**, which is covered in detail in Section 6.

APIVerify was introduced in 2020 as an internal tool for the team building MockBridge, MQS's service virtualization product, and was later generalized into a standalone framework used across most MQS API teams. Documentation and release notes for APIVerify are published at <https://docs.meridianqs.com/apiverify/>.

1.1 Why Contract Testing Matters

Traditional end-to-end tests catch problems only when a full user journey breaks, which is often too late and too expensive to diagnose quickly. Contract tests catch a much narrower and more specific class of problem — an API silently changing shape — much earlier, typically before a consumer application ever notices. This is especially valuable in organizations where the team consuming an API is different from the team producing it, since a contract acts as an explicit, checkable agreement between the two rather than an informal understanding that can quietly drift out of sync as both sides evolve independently over time.

2. Core Concepts

Before installing APIVerify, it helps to understand three core concepts that recur throughout this handbook: **Contract Definitions**, **Endpoint Groups**, and **Baselines**. These three concepts build on one another, and later sections assume familiarity with all three.

2.1 Contract Definitions

A Contract Definition is a YAML or JSON file describing exactly what an endpoint is expected to return. A minimal contract definition looks like this:

```
contract: get-user-profile
endpoint: GET /v1/users/{id}
expected_status: 200
response_schema:
  type: object
  required: [id, email, created_at]
  properties:
    id: { type: string }
    email: { type: string }
    created_at: { type: string, format: date-time }
```

Contract Definitions are stored alongside application code, typically in a `contracts/` directory, and are versioned in source control just like any other artifact.

2.2 Endpoint Groups

Related endpoints are organized into an **Endpoint Group**, which allows a team to run verification against a logical slice of an API rather than the entire surface at once. A typical service might define an Endpoint Group called `users`, containing every contract related to user profile management, and a separate group called `billing`, containing contracts related to invoicing and payments.

Endpoint Groups matter for two practical reasons: they let teams parallelize verification runs across CI jobs, and they let a team scope a single verification run to only the part of the API they are actively changing, rather than re-checking an entire service on every commit.

2.3 Baselines

APIVerify uses the word **Baseline** in two related but distinct senses, and this handbook is careful to distinguish them because confusing the two is a common source of misconfiguration.

A **Contract Baseline** is the accepted, known-good version of a Contract Definition — effectively a snapshot of what the contract looked like the last time a human reviewer approved it. A **Performance Baseline** is a separate, optional snapshot of an endpoint's typical response time, used only when latency checks are enabled. Section 5 (Baseline Sync) covers the Contract Baseline in depth; Performance Baselines are a smaller, opt-in feature described briefly in Section 7.

3. Installation and Setup

APIVerify is distributed as a Python package, consistent with the rest of the MQS testing toolchain.

```
pip install apiverify==2.3.0
```

After installation, initialize a project:

```
apiverify init my-api-contracts
```

This creates the following structure:

```
my-api-contracts/  
  apiverify.yaml  
  contracts/  
    users/  
    billing/  
    baselines/
```

3.1 Minimum Requirements

Requirement	Minimum Version
Python	3.9
Operating System	Windows, macOS, Linux
Memory	2 GB RAM
Network Access	Outbound HTTPS to target API

NOTE: Unlike FalconTest, which at the time of the FalconTest 4.2 guide supported only Windows and macOS, APIVerify 2.3 supports Linux as well. This is a deliberate platform difference between the two products, not an inconsistency — each product's supported platforms should be checked independently rather than assumed to match.

3.2 Initial Configuration

On first run, APIVerify prompts for a target base URL and, optionally, an authentication token. These are stored in `apiverify.yaml`:

```
base_url: https://api.example.com  
auth:  
  type: bearer  
  token_env_var: API_TOKEN  
verification:  
  timeout_ms: 8000  
retry:  
  max_attempts: 2
```

The `verification.timeout_ms` and `verification.retry.max_attempts` values shown above were the defaults at the time APIVerify 2.0 was released. As covered later in Section 7, these defaults were revised in version 2.3.

4. Verification Modes

APIVerify can run in one of two verification modes: **Strict Mode** and **Lenient Mode**. The choice of mode changes how APIVerify treats fields that appear in a live response but are not declared in the contract.

4.1 Strict Mode

In Strict Mode, any field present in the live response that is not explicitly declared in the Contract Definition's schema is treated as a verification failure. Strict Mode is the more conservative option: it guarantees that a contract fully and exactly describes a response shape, with nothing undocumented slipping through. Teams that treat their contracts as the authoritative source of truth for API documentation generally prefer Strict Mode, since any drift — even the addition of a harmless new field — is surfaced immediately.

4.2 Lenient Mode

In Lenient Mode, extra fields in the live response that are not declared in the contract are ignored, and only the fields that are declared are checked for correctness. Lenient Mode is better suited to APIs that are actively evolving, where new fields are added frequently and a team does not want every addition to trigger a failed verification run. The tradeoff is that Lenient Mode can allow genuine, unintentional schema drift to go unnoticed for longer, since only declared fields are ever checked.

4.3 Choosing a Mode

Aspect	Strict Mode	Lenient Mode
Undeclared fields	Fail verification	Ignored
Best for	Stable, well-documented APIs	Rapidly evolving APIs
False positive rate	Higher	Lower
Drift visibility	Immediate	Delayed until a declared field changes

TIP: A common pattern at MQS is to run new Endpoint Groups in Lenient Mode for their first few weeks, then switch to Strict Mode once the contract has stabilized. This is covered in more detail in Section 8, where it intersects with MockBridge stub generation.

5. Baseline Sync

Baseline Sync is the process by which APIVerify reconciles a Contract Definition's Baseline with the current state of the live API, and it is one of the concepts in this handbook most likely to span a page boundary, so read this section together with Section 5 (continued) below.

The Baseline Sync process runs in four steps:

- 1 **Fetch** — APIVerify sends a request to the live endpoint using any example parameters defined in the contract.
- 2 **Compare** — the live response is compared against the current Contract Baseline, field by field, using the verification mode (Strict or Lenient) configured for that Endpoint Group.
- 3 **Flag** — if a discrepancy is found, APIVerify does not automatically update the Baseline. Instead, it flags the discrepancy for human review, since an unreviewed automatic update could silently accept a real regression as if it were an intentional change.

5.1 Baseline Sync (continued)

- 4 **Approve or Reject** — a reviewer examines the flagged discrepancy and either approves it, which updates the Contract Baseline to match the new live response, or rejects it, which leaves the Baseline unchanged and treats the live response as a genuine regression that needs to be fixed in the API itself rather than in the contract.

This four-step approval workflow is intentional: APIVerify treats the Contract Baseline as something that should only ever change through explicit human approval, never automatically. This is the single most important design decision in APIVerify's Baseline Sync process, because it is what prevents contract drift from silently becoming the new accepted normal simply because nobody objected to it.

Baseline Sync runs are triggered manually via `apiverify sync`, or automatically on a schedule if `sync.auto_schedule` is enabled in `apiverify.yaml`. MQS's internal guidance recommends running Baseline Sync manually rather than on an automatic schedule for any Endpoint Group still in Strict Mode, precisely because Strict Mode discrepancies are more likely to represent a genuine problem worth a human's immediate attention rather than routine drift.

6. Drift Detection

Drift Detection is the umbrella term for APIVerify's continuous checking of live API responses against Contract Baselines, and it is powered by the same Fetch-Compare-Flag mechanism described in Section 5. The difference between Baseline Sync and Drift Detection is one of intent rather than mechanism: Baseline Sync is run deliberately, when a team expects the contract may need updating, while Drift Detection runs continuously (or on every CI build) specifically to catch *unexpected* changes.

6.1 Drift Severity Levels

Not all drift is equally serious. APIVerify classifies detected drift into three severity levels.

Severity	Meaning	Example
Info	A new, undeclared field appeared (Lenient Mode only)	A metadata field added to a response
Warning	A declared field's type changed in a backward-compatible way	An integer field now also accepts null
Critical	A declared field was removed, renamed, or changed status code	The email field was removed entirely

WARNING: Critical drift always fails a verification run regardless of whether the Endpoint Group is running in Strict or Lenient Mode. The Strict/Lenient distinction only affects how undeclared fields are treated — it does not affect how declared-field regressions are treated.

7. Configuration Reference

Full configuration for APIVerify lives in `apiverify.yaml`.

Key	Type	Default (v2.0)	Default (v2.3)	Description
<code>verification.timeout_ms</code>	integer	8000	10000	Timeout in ms
<code>verification.retry.max_attempts</code>	integer	2	3	Retry attempts
<code>verification.mode</code>	string	strict	strict	Default mode for new groups
<code>sync.auto_schedule</code>	string	disabled	disabled	Cron for auto Baseline Sync
<code>performance_baseline.enabled</code>	boolean	false	false	Enables Performance Baseline

NOTE: APIVerify 2.3 raised both `verification.timeout_ms` and `verification.retry.max_attempts` from their 2.0 defaults. Any configuration reference to the timeout or retry count elsewhere in MQS documentation should be understood as describing the version in effect at the time it was written; this table reflects the current, version 2.3 defaults unless otherwise noted.

7.1 Performance Baselines

When `performance_baseline.enabled` is set to `true`, APIVerify additionally records response-time percentiles for each endpoint and flags a Warning-level drift event if p95 latency exceeds the recorded Performance Baseline by more than 50%. Performance Baselines are stored separately from Contract Baselines and are not subject to the Baseline Sync approval workflow described in Section 5, since a temporary latency spike does not require the same deliberate human review as a schema change.

8. Integration with MockBridge and FalconTest

APIVerify is designed to work alongside two other MQS products.

MockBridge can generate a service virtualization stub directly from a Contract Definition, which allows consumer teams to develop and test against a realistic mock of an API before the real implementation is finished. When a contract's Baseline is updated through Baseline Sync (Section 5), MockBridge stubs generated from that contract can be regenerated automatically to stay in sync, avoiding a situation where a consumer team is developing against a stub that no longer matches the real API's approved contract.

FalconTest can invoke APIVerify checks as a step within a broader end-to-end test suite, using the `apiverify_check` fixture. This lets a FalconTest suite fail fast on a contract violation before proceeding to a slower, full end-to-end scenario that depends on that same endpoint behaving correctly. Verification jobs invoked this way run on FalconTest's Execution Cluster (see the FalconTest Automation Framework Guide, Section 2) rather than as a separate standalone process.

TIP: For CI pipelines that run on every commit, run new or actively-changing Endpoint Groups in Lenient Mode through APIVerify directly, and reserve FalconTest's `apiverify_check` fixture — with the target Endpoint Group set to Strict Mode — for the smaller set of stable, contract-complete APIs where any drift at all should block a merge.

9. Reporting and Dashboards

APIVerify publishes verification results in the same three formats supported by FalconTest — HTML, JSON, and JUnit XML — for consistency across the MQS toolchain, plus one format unique to APIVerify.

Format	Description
HTML	Interactive report with per-endpoint drift history
JSON	Machine-readable output for downstream tooling
JUnit XML	Compatible with BuildFlow's test result visualization
Drift Timeline	Web dashboard showing drift events across all Endpoint Groups over time, available at https://apiverify.internal.meridianqs.com/timeline

The Drift Timeline dashboard is the primary tool teams use to spot slow, gradual drift that individual verification runs might not make obvious in isolation, since it aggregates results across many runs rather than showing only the outcome of the most recent one.

10. Best Practices

- Start new Endpoint Groups in Lenient Mode, and graduate to Strict Mode once the contract has stabilized.
- Never enable `sync.auto_schedule` for an Endpoint Group running in Strict Mode; require manual review instead.
- Treat Critical-severity drift as a build-blocking failure regardless of verification mode.
- Enable Performance Baselines only for endpoints with well-understood, stable latency characteristics, since a noisy endpoint will generate frequent false-positive Warning events.
- When integrating with FalconTest via `apiverify_check`, reserve Strict Mode targets for genuinely stable APIs to avoid frequent, low-value suite failures.

11. Glossary

Term	Definition
Contract Definition	A YAML or JSON file describing an endpoint's expected request/response shape
Endpoint Group	A logical grouping of related contracts, used to scope verification runs
Contract Baseline	The accepted, known-good version of a Contract Definition
Performance Baseline	An optional snapshot of an endpoint's typical response time
Drift	Any deviation between a live API response and its Contract Baseline
Drift Detection	The continuous process of checking live responses against Contract Baselines
Baseline Sync	The deliberate, human-reviewed process of updating a Contract Baseline
Strict Mode	Verification mode that fails on any undeclared field
Lenient Mode	Verification mode that ignores undeclared fields

12. Questions to Try

The following questions are provided to help you explore this document using your own RAG pipeline. No answers are provided here — try retrieving and generating answers yourself.

- 1 What is a Contract Definition in APIVerify?
- 2 How does the Baseline Sync approval workflow prevent silent contract drift?
- 3 Compare Strict Mode and Lenient Mode.
- 4 Which verification mode should a team use when integrating a new, rapidly-changing Endpoint Group with FalconTest's `apiverify_check` fixture?
- 5 What is the current default value of `verification.retry.max_attempts` in APIVerify 2.3?
- 6 What is a Baseline in APIVerify — the Contract Baseline or the Performance Baseline?
- 7 Does APIVerify support GraphQL APIs?
- 8 What operating systems does APIVerify 2.3 support?
- 9 What are the three Drift severity levels, and which one always fails a verification run?
- 10 How does APIVerify's Drift Detection differ from Baseline Sync?